Data Science
Machine Learning

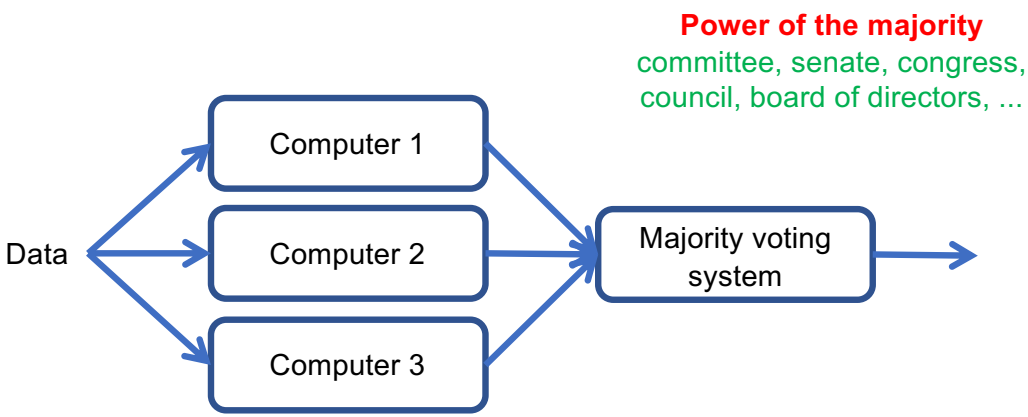
Today's Topic – Ensemble Models
Classifiers
Random Forest and Boosting

In this file:

- Bagging (averaging, voting) vs Boosting (incremental correction)
- Ensemble Learning, Trees, Random Forest vs Boosing
- Binary Classification - Islands - when linear separation doesn't work
- Decision Tree - and how to build it
- Building Tree - Gini Impurity
- Building Tree - Smallest Entropy
- Building Tree – CHAID
- Random Forest – authors
- Bagging – Boosting – Stacking
- Random Forest diagram
- Bootstrap Sampling
- Boosting Authors
- Gradient Descent, Gradient Boosting, Validation
- Ada Boost diagram
- Misc. links

Idea:

make a reliable system from unreliable blocks using single-shot democratic voting



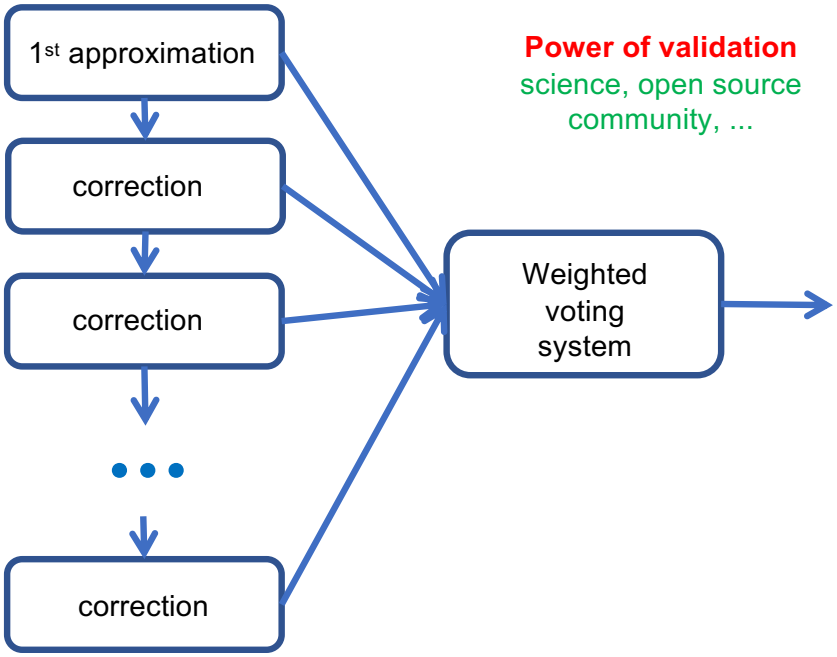
Power of the majority
committee, senate, congress, council, board of directors, ...

Bagging

Wrong bias is not removed, it is simply averaged

Idea:

get to correct answer by making multiple corrective steps



Power of validation
science, open source community, ...

Wrong bias is removed using correction and validation

Boosting

Ensemble Learning

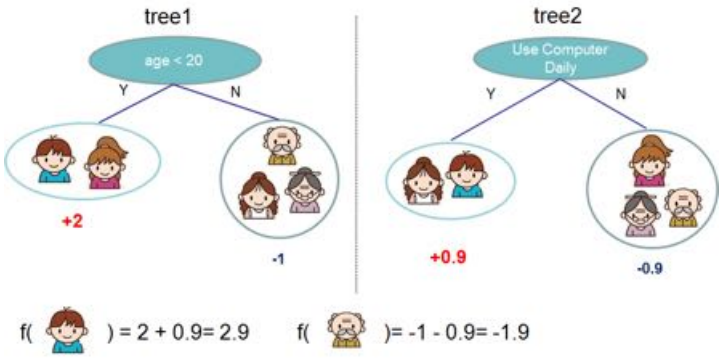
Idea - train several models to do predictions.
Then combine predictions of these models to get a better predictor.

It is possible to combine any types of models into one predictor.
But commonly people combine variations of similar models, like **decision trees** models.

Two most famous approaches are:
- **"Random Forest"**
- **"Gradient Boosted Trees"**

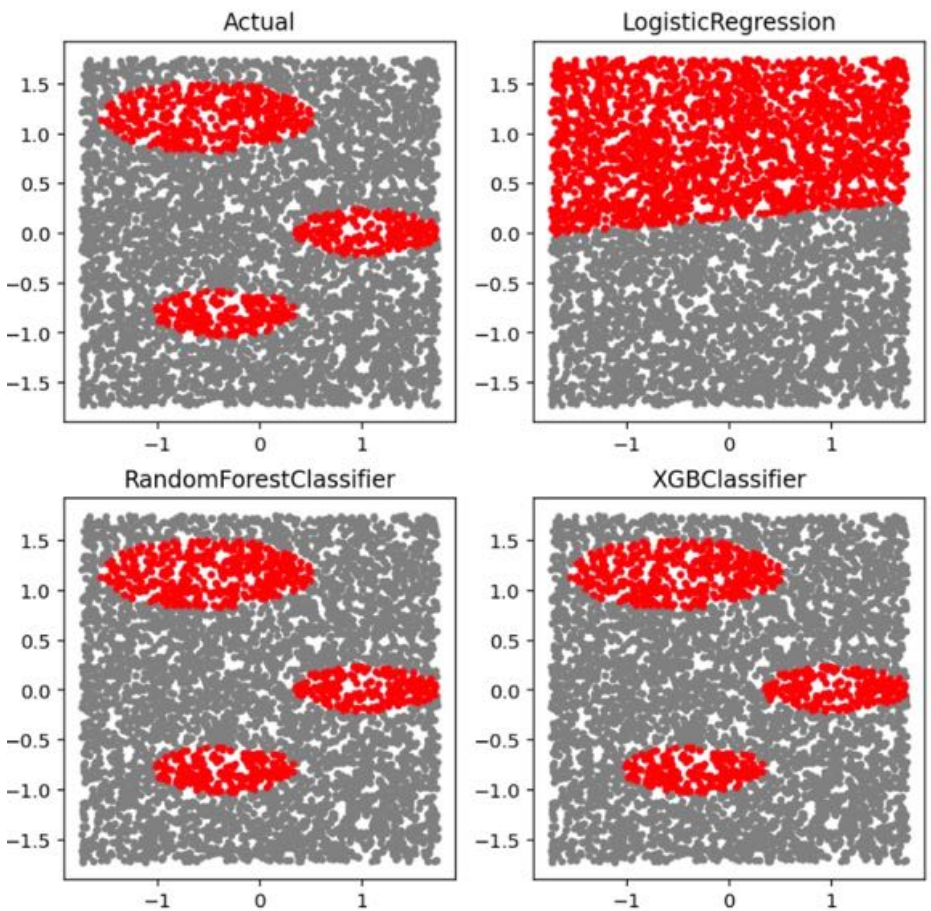
Under the hood these models combine 100 ... 200+ similar trees.

The differences between algorithms:
- how you select the data to train those trees
- how you combine the predictions of these trees

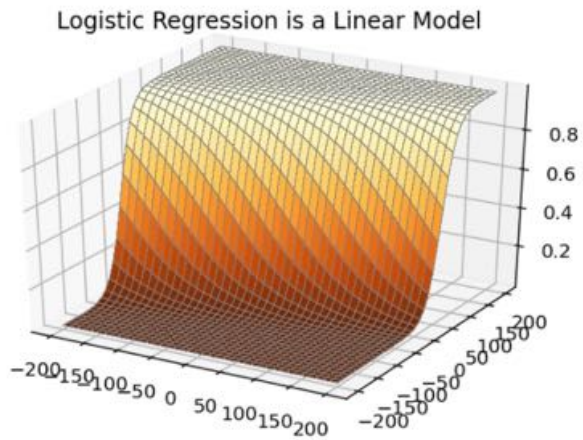


Binary Classification Problem

Example – two classes: red and grey.



Ensemble Tree Classifiers succeed where Linear Classifiers fail

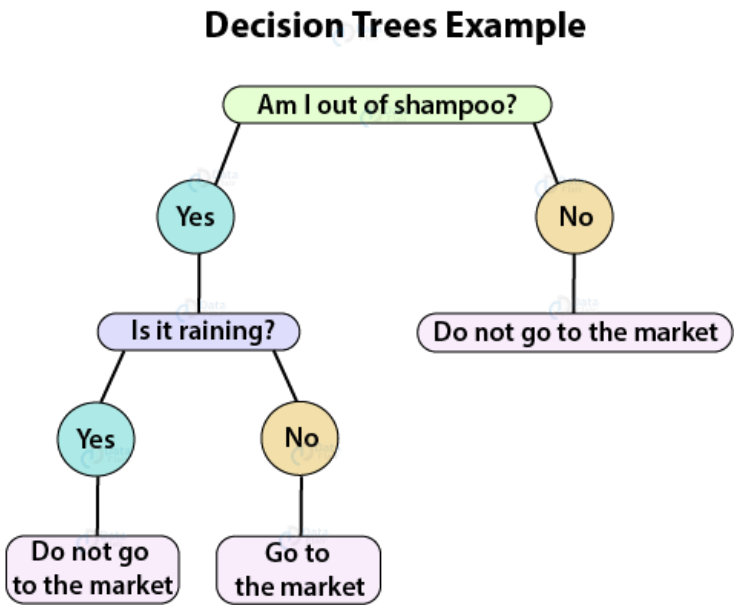


Ensemble tree classifiers
(Random Forest , XGBoost)
can solve complicated problems

Decision Tree (DT)

DT goes from observations about an item (represented in the branches) to conclusions about the item's target value (represented in the leaves).
DT can do classification or regression.
DTs are intuitive and easily interpretable.

Algorithms for constructing DTs usually work top-down, by choosing a variable at each step that best splits the set of items.
We test all possible splits (or some subset) to decide how to split.
Simplicity is best, so we want to keep our tree small.
To do so, at each step we should choose the split that results in the purest daughter nodes.



Decision Tree Building Algorithms

- **ID3 (Iterative Dichotomiser 3)** – uses smallest entropy metrics
- **C4.5** – successor of ID3
- **CART (Classification And Regression Tree)** – uses **Gini metrics**
- **CHAID (CHi-squared Automatic Interaction Detector)**. Performs multi-level splits when computing classification trees.
- **MARS**: extends decision trees to handle numerical data better.
- **Conditional Inference Trees**. Statistics-based approach that uses non-parametric tests as splitting criteria, corrected for multiple testing to avoid overfitting. This approach results in unbiased predictor selection and does not require pruning.

Gini Impurity

Gini Impurity (I_G) - metrics used by CART method for making splitting decisions on categorical features.

Gini impurity is a measure of how often a randomly chosen element from the set would be **incorrectly labeled** if it was randomly labeled according to the distribution of labels in the subset.

Gini max value is 0.5, and min value is 0 (when all cases in the node fall into a single target category).

If we have a set of items with J classes {1,2,..,J}, and p_i is the fraction of items of i-th class, then it can be shown that :

$$I_G = 1 - \text{sum}(p_i^2)$$

$$I_G(p) = \sum_{i=1}^{J}\left(p_i \sum_{k \neq i} p_k\right) = \sum_{i=1}^{J} p_i(1-p_i) = \sum_{i=1}^{J}(p_i - p_i^2) = \sum_{i=1}^{J} p_i - \sum_{i=1}^{J} p_i^2 = 1 - \sum_{i=1}^{J} p_i^2$$

fraction of incorrectly labeled

If you look at the documentation for the DecisionTreeClassifier class in scikit-learn, you'll see something like this for the criterion parameter:

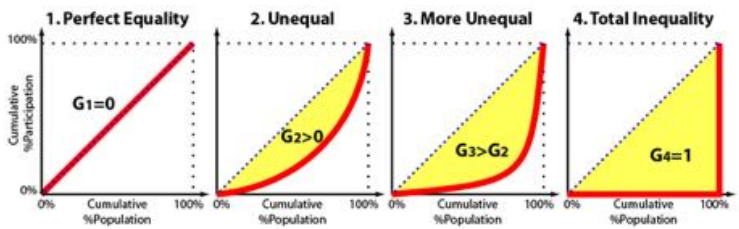
criterion : *string, optional (default="gini")*

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain.

When training a decision tree, the best split is chosen by maximizing the **Gini Gain**, which is calculated by subtracting the weighted impurities of the branches from the original impurity - https://victorzhou.com/blog/gini-impurity/

Corrado Gini
1884-1965

Name "Gini" comes from economics.
Corrado Gini - Italian statistician and sociologist.
He has published his paper Variability and Mutability in 1912.
He has introduced a **Gini coefficient**, sometimes called the Gini index or Gini ratio.
It is a measure of statistical dispersion intended to represent the **income inequality** or wealth inequality within a nation or any other group of people.
The coefficient ranges from 0 (or 0%) to 1 (or 100%), with 0 representing perfect equality and 1 representing perfect inequality.



Smallest Entropy (Largest Information Gain)

ID3 (ID.3 = Iterative Dichotomiser 3)

- an algorithm invented by Ross Quinlan (1986)
- used to generate a decision tree from a dataset
- **splits data by an attribute that has smallest average entropy of the split (or largest information gain)**

C4.5 is an extension of ID3, used for classification, has following advantages:

- handles both continuous and discrete attributes (creates a threshold and then splits the list)
- handles missing attributes data (simply not using them in entropy calculations).
- handles attributes with differing costs
- pruning trees after creation (attempts to remove inefective branches or replace them with leaf nodes).

C5.0 algorithm - improved C4.5:

- https://www.rulequest.com/see5-info.html
- https://www.rulequest.com/download.html

- much-much faster
- less memory usage
- builds smaller decision trees
- support for boosting
- support for weighting
- automating winnowing (removing not effective attributes)

John Ross Quinlan
Inventor of canonical C4.5 and ID3 algorithms. Currently running his company **RuleQuest Research**.

- https://www.rulequest.com/Personal/
- https://www.rulequest.com/

$$Entropy = \sum_j -p_j log_2 p_j$$

Pure node : entropy = 0

Information Gain = 1 - Entropy

CHAID – Chi-Squared Automatic Interaction Detection

Decision Tree Building Algorithm

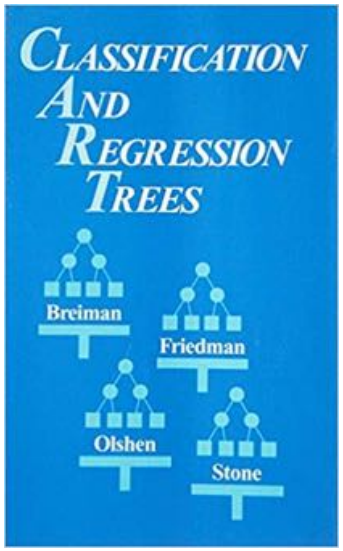
CHAID - Chi-Squared Automatic Interaction Detection.

It was originally proposed by Gordon V. Kass in 1980
- https://en.wikipedia.org/wiki/Chi-square_automatic_interaction_detection

This algorithm used only for categorical variables.

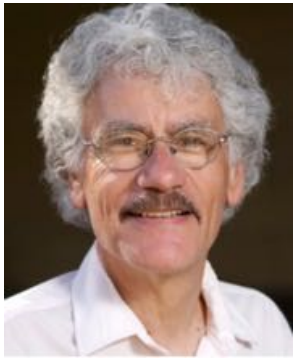
In practice, CHAID is often used in the context of direct marketing to select groups of consumers and predict how their responses to some variables affect other variables.

Good explanation of CHAID vs CART:
https://towardsdatascience.com/clearly-explained-top-2-types-of-decision-trees-chaid-cart-8695e441e73e



Classification and Regression Trees
(1984) by BFOS (BFOS = Breiman, Friedman, Olshen, Stone)

Leo Breiman
UC Berkeley
Statistics
1928-2005



Jerome H. Friedman
Statistics
UC Berkeley,
Stanford
1939-

Richard A. Olshen
Biostatistics
UC Berkeley,
Stanford
1942-

Charles J. Stone
Math, Probability
UCLA,
UC Berkeley
1936-2019

FORTRAN
Random Forest
2002-2003

Adele Cutler
UC Berkeley, then Utah State University.
Wrote original Random Forest in Fortran with Leo Breiman in 2002-2003
https://www.stat.berkeley.edu/~breiman/RandomForests/cc_software.htm

Random Decision Forests
(1995) by Tin Kam Ho

Random Decision Forests

Tin Kam Ho
AT&T Bell Laboratories
600 Mountain Avenue, 2C-548C
Murray Hill, NJ 07974, USA

Abstract

Decision trees are attractive classifiers due to their high execution speed. But trees derived with traditional methods often cannot be grown to arbitrary complexity for possible loss of generalization accuracy on unseen data. The limitation on complexity usually means sub-optimal accuracy on training data. Following the principles of stochastic modeling, we propose a method to construct tree-based classifiers whose capacity can be arbitrarily expanded for increases in accuracy for both training and unseen data. The essence of the method is to build multiple trees in randomly selected subspaces of the feature space. Trees in different subspaces generalize their classification in complementary ways, and their combined classification can be monotonically improved. The validity of the method is demonstrated through experiments on the recognition of handwritten digits.

Our study shows that this difficulty is not intrinsic to tree classifiers. In this paper we describe a method to overcome this apparent limitation. We will illustrate the ideas using oblique decision trees which are convenient for optimizing training set accuracy. We begin by describing oblique decision trees and their construction, and then present the method for increasing generalization accuracy through systematic creations and use of multiple trees. Afterwards, experimental results on handwritten digits are presented and discussed.

2 Oblique Decision Trees

Binary decision trees studied in prior literature often use a single feature at each nonterminal (decision) node. A test point is assigned to the left or right branch by its value of that feature. Geometrically this

Tin Kam Ho
at Bell Labs in NJ.
then AI at IBM (since 2014)

Ensemble Learning Methods with Decision Trees

A group of "weak learners" come together to form a "strong learner"

Bagging

reduces variance (averages)
does NOT reduce bias error

Random Forest – parallel independent learners, democratic equal-weight voting.

Note - **Random Forest** is a specific case of **Bagging** = Bootstrap Aggregating

Bootstrap = data sampling with replacement
Aggregating = voting

Each learner gets a random subset of data (rows) and a random subset of features (columns).

Each tree is trained independently.

Each tree then can make its prediction.
Then overall decision is made by averaging decisions of individual trees.

Boosting

reduces bias error
by sequential corrective learning

Boosted Trees – trees trained sequentially.

Train 1st tree

Train 2nd tree concentrating on fixing errors left after 1st tree, combine trees (weighted voting)

Train 3rd tree concentrating on fixing errors left after 1st and 2nd trees, etc.

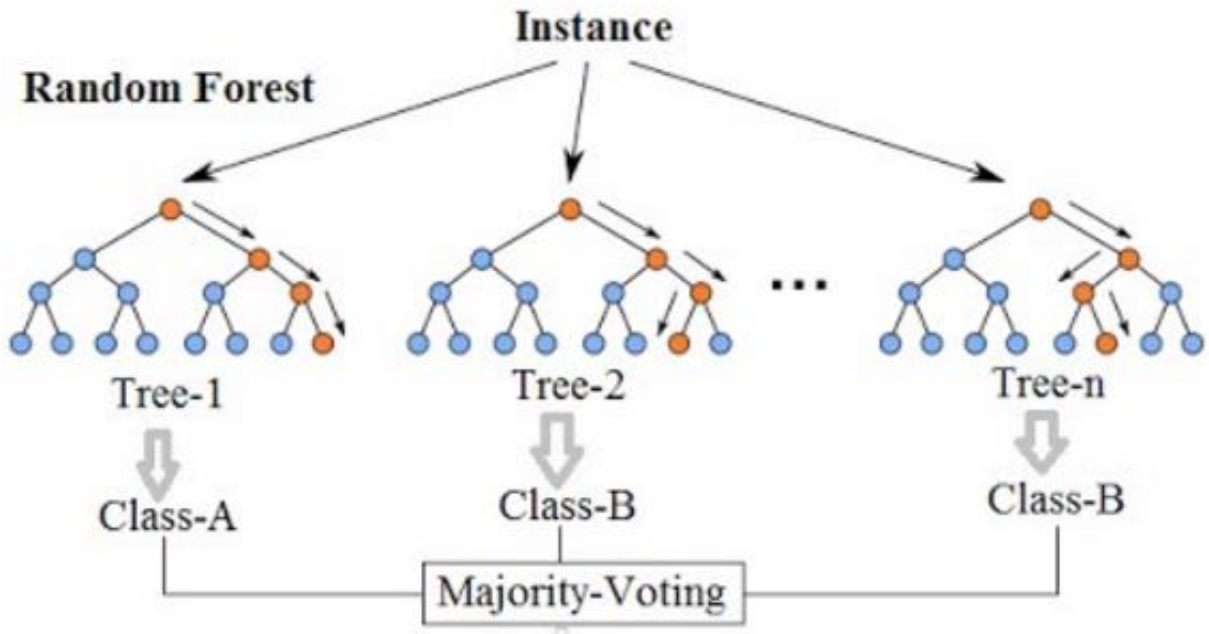
Continue boosting accuracy by adding trees.

Stacking

improves predictive force
(combines "skills" of different models)

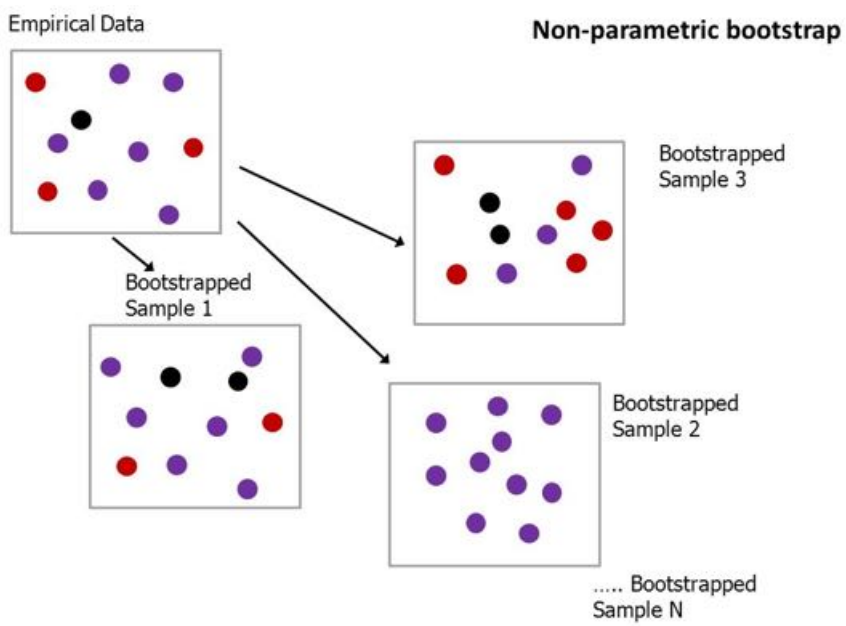
Stacking (Meta-learner) - train several different models, and then train model on top of their outputs
- https://www.kdnuggets.com/2017/02/stacking-models-imropved-predictions.html

Random Forest Simplified



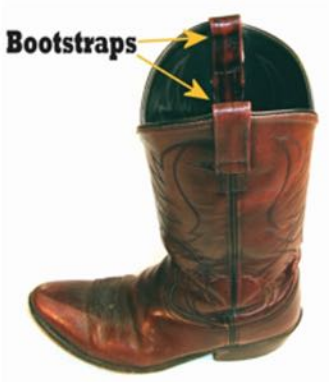
Bootstrap Sampling

method of preparing a set of data samples by sampling them (with replacement) from original data set



Another meaning of the word bootstrap:

"pull oneself over a fence by one's bootstraps"

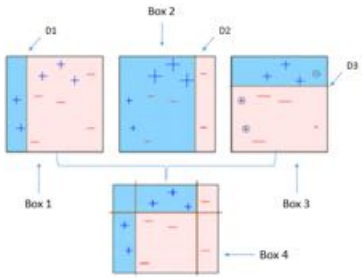


Boosting Algorithms

Strength of Weak Learners

AdaBoost (1995)

Yoav Freund & Robert Schapire
(2003 Gödel Prize).



XGBoost (2014)

more flexible and optimized
Tianqi Chen (was a Ph.D. student)

Light GBM (2016, Microsoft)

fast-performing, distributed
(GBM = Gradient Boosting Machine)

Microsoft

CatBoost (2017, Yandex)

big data, categorical data, multiple GPUs.

Yandex

Leo Breiman
UC Berkeley

Jerome H. Friedman
UC Berkeley,
Stanford

From Wikipedia, etc:

The idea of gradient boosting originated in the observation by **Leo Breiman** that boosting can be interpreted as an optimization algorithm on a suitable cost function (**1997**, Berkeley).

Explicit regression gradient boosting algorithms were subsequently developed by **Jerome H. Friedman** (**1999**, Berkeley).

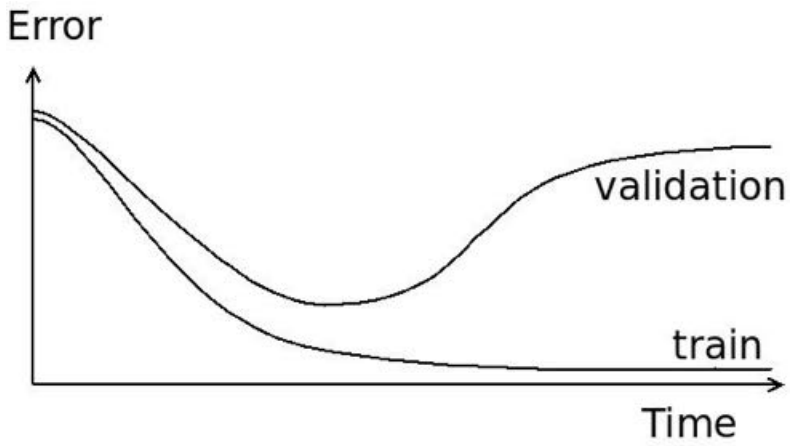
... simultaneously with the more general functional gradient boosting perspective of **Llew Mason, Jonathan Baxter, Peter Bartlett and Marcus Frean** (**1999**).

Stochastic gradient boosting - by Jerome H.Friedman, **2002** – adding random sampling to gradient boosting algorithm improves speed and accuracy.

Gradient Boosting

is named so because on each steep
it is improving (boosting) the decision
by decreasing the error (Gradient Descent)

"validation data set" – used in boosting to figure out when to stop iterations to avoid overfitting

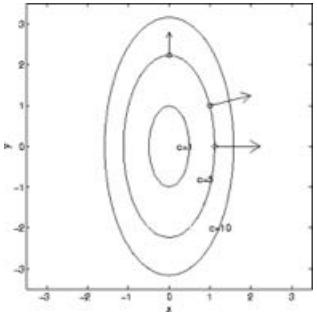


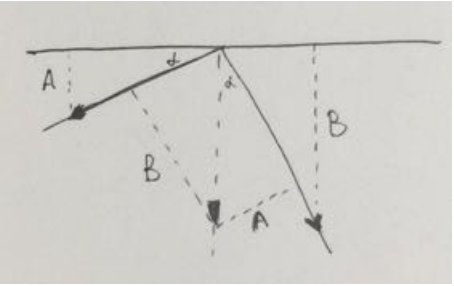
Gradient Descent

Gradient vector is
perpendicular to level curves

Understand gradient vector

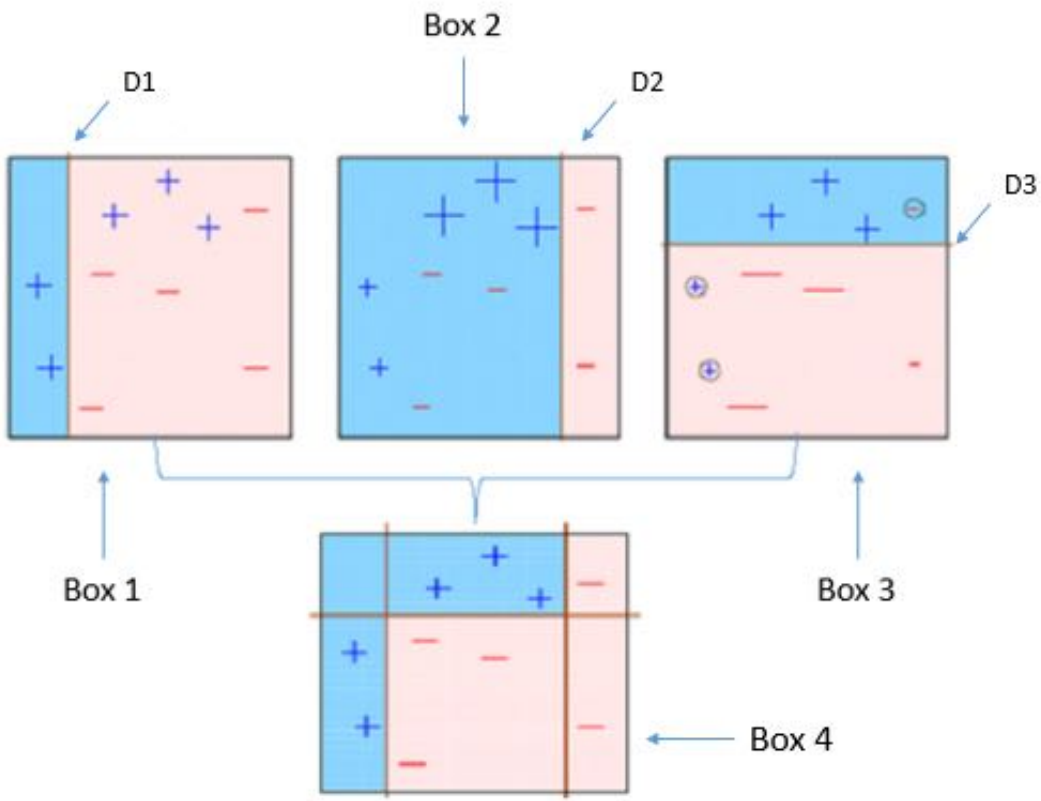
$$\nabla\phi = \begin{bmatrix} \frac{\partial\phi}{\partial x} \\ \frac{\partial\phi}{\partial y} \\ \frac{\partial\phi}{\partial z} \end{bmatrix}$$





AdaBoost

First Boosting Algorithm
(1995) by Freund and Schapire



Gradient boosting trees model is originally proposed by Friedman et al.
He has coined the term GBM = *Gradient Boosting Machine*
"Gradient Boosting" - *Greedy Function Approximation: A Gradient Boosting Machine*, by Friedman (1999)
- https://statweb.stanford.edu/~jhf/ftp/trebst.pdf

XGBoost was created by Tianqi Chen. It is a library implementing GBM – and optimized for trees.
"XG" in XGBoost = "Extreme Gradient Boosting".
Tianqi Chen - Introduction to Boosted Trees (2016)
- http://homes.cs.washington.edu/~tqchen/pdf/BoostedTree.pdf
- https://xgboost.readthedocs.io/en/latest/model.html

XGBoost
- uses "Regularized Gradient Boosting" model which prevents overfitting and gives better performance
- uses sparse matrices with sparsity aware algorithms to reduce memory and increase performance
- better support for multicore parallel processing - reduces training time

2015 - DART: Dropouts meet Multiple Additive Regression Trees
- https://arxiv.org/abs/1505.01866
- http://zhanpengfang.github.io/418home.html
- https://www.analyticsvidhya.com/blog/2016/03/complete-guide-parameter-tuning-xgboost-with-codes-python/
- https://www.quora.com/What-is-the-difference-between-the-R-gbm-gradient-boosting-machine-and-xgboost-extreme-gradient-boosting

- https://en.wikipedia.org/wiki/Gradient_boosting
- https://en.wikipedia.org/wiki/Xgboost

==========
Leo Breiman, Jerome Friedman, CART, BFOS on youtube:
- https://www.youtube.com/watch?v=t8ooi_tJHSE – 2 min, Working with Leo Breiman on Random Forests, Adele Cutler
- https://www.youtube.com/watch?v=JIXeMj1zwVU – 5min, CART
- https://www.youtube.com/watch?v=-Ju0Czc9GmA – 4min, The Birth of BFOS: Breiman, Freidman, Olshen and Stone
- https://www.youtube.com/watch?v=XaeQo1KSmjA – 4min, CART and Cross-Validation, Data Mining
- https://www.youtube.com/watch?v=8hupHmBVvb0 – 4min, Jerome Friedman, Data Mining Software Founding Father